Monads and Algebraic Effects

Conner Rose (cnrose@umich.edu)
University of Michigan

April 20, 2026

Modern programming languages must account for a wide range of computational effects, including mutable state, exceptions, I/O, and nondeterminism. While these features are required for real-world programs, they complicate reasoning about such programs. A central challenge in programming language theory is therefore to develop abstractions that allow such effects to be described, composed, and reasoned about in a principled way.

One of the most influential approaches to this problem is the use of monads, introduced to programming language semantics by Moggi [1]. Monads provide a framework for structuring effectful computations, allowing programmers to sequence computations while abstracting over the the underlying effects. This abstraction has been highly successful in practice, particularly in functional programming languages such as Haskell [2]. However, monads can make it harder to see what effects a program is actually using, since they only provide a general way to sequence computations rather than exposing the specific operations of each effect.

An alternative approach, developed more recently, is to describe computational effects in "algebraic" terms, where effects are specified by collections of operations together with equations that govern their behavior [3], [4]. This leads to the theory of algebraic effects and handlers. This shift provides a more modular and compositional framework for reasoning about effects, since effects can be specified independently and combined without requiring a fixed global composition strategy.

Monadic Foundations of Computation

The introduction of monads into programming language theory provides a foundational framework for modeling effectful computation [1]. By distinguishing between values and computations, this approach captures the idea that effectful programs do more than simply return results, they also perform actions during evaluation.

This perspective enables a compositional semantics in which effectful programs can be interpreted systematically. In particular, the structure provided by monads supports the sequencing of computations. This abstraction laid the groundwork for later developments.

Monads in Practice

The monadic view of computation proved to be a powerful programming abstraction [5]. By encoding common effects (state, exceptions, and I/O, etc.), monads allow programmers to write effectful code in a purely functional style. This approach preserves compositionality while making effects explicit in the structure of programs.

The adoption of monads in Haskell demonstrates the practical impact of this idea [2]. By standardizing monadic interfaces, Haskell enables programmers to build and combine effectful computations in a consistent way. However, this success also highlights certain limitations. In particular, combining multiple effects often requires additional machinery, such as monad transformers, and the monadic structure can obscure the underlying operations that define each effect.

Algebraic Effects and Handlers

A different approach considers computational effects as algebraic structures. Rather than encoding effects indirectly through monads, this approach describes them in terms of operations and

equations [3], [4]. For example, state can be characterized by operations for reading and writing values, together with laws that describe how these operations interact. Concretely, if we wanted to model a single memory cell holding type `value`, this could be given by two operations:

```
get : unit  → value
set : value → unit
```

along with the following equations (leaning on do-notation for brevity):

```
do { set(x); get()  } = x
do { set(x); set(y) } = set(y)
```

These equations are essential because the operations alone do not determine the behavior of the effect. Without equations, `get` and `set` would be arbitrary operations with no guarantee that they behave like a coherent memory cell. The equations specify how operations interact, ensuring, for example, that a value written with `set` is the one subsequently returned by `get`, and that later writes overwrite earlier ones. In this sense, an algebraic effect is not just a collection of operations, but a theory consisting of operations together with laws that constrain their behavior. Different implementations (handlers) must satisfy these laws, which ensures that all interpretations of the effect exhibit the same essential semantics.

This algebraic viewpoint leads naturally to the notion of handlers, which define how effectful operations are interpreted [3]. By separating the definition of operations from their implementation, handlers provide a flexible and modular mechanism for composing effects. Different handlers can give different meanings to the same operations, enabling a high degree of reuse and abstraction.

This framework addresses key limitations of monadic approaches, particularly in terms of modularity. Effects can be combined without requiring a fixed global structure, and their behavior can be customized locally through handlers. Bauer and Pretnar show that this approach can be integrated into a full programming model, supporting modular and compositional reasoning about effects [4]. As a result, algebraic effects offer a more fine-grained and extensible way of structuring effectful computation.

Effects in Modern Language Design

The algebraic approach to effects has influenced the design of modern programming languages. In particular, languages such as Koka incorporate effect information directly into the type system, allowing the effects of a program to be tracked and analyzed statically [6]. This provides greater transparency and enables more precise reasoning about program behavior.

By combining algebraic effects with advanced type system features such as row polymorphism, Koka supports flexible composition of effects while maintaining strong guarantees about program correctness [6]. This demonstrates that theoretical advances in the understanding of effects can lead to practical improvements in language design.

The algebraic view of effects connects directly to row-polymorphic effect systems such as Koka. As mentioned, in an algebraic theory, an effect is described by a set of operations, together with equations governing their behavior. Effect rows can be understood as tracking exactly which such operations a computation may invoke. For example, an effect row containing `state` or `io` corresponds to allowing the associated operations in the algebraic theory.

Row polymorphism then allows functions to remain generic over these sets of operations. For example, a row polymorphic `map` function will have type

$$\texttt{map} : \forall \alpha\beta\mu.(\texttt{list}\ (\alpha), \alpha \to \mu\beta) \to \mu\ \texttt{list}\ (\beta)$$

where μ corresponds to the effects of the function we're applying to the `list`. The return type of `map` indicates that whatever effects the function may have are propagated to the overall computation. In particular, `map` itself does not introduce any new effects; it simply sequences the effects of its argument. As a result, if the function performs I/O, the entire `map` computation has an `io` effect, and if it performs stateful updates, the resulting computation reflects those state effects. This allows map to be reused uniformly across many different effects, while still precisely tracking them in the type system.

Conclusion and Open Questions

The evolution from monadic to algebraic approaches reflects a broader shift in how computational effects are understood. While monads provide a powerful and widely used abstraction [1], [5], algebraic effects reveal that these structures can be derived from more fundamental descriptions based on operations and equations [3], [4]. This shift leads to more modular and expressive systems for structuring programs.

Despite this progress, several open questions remain. One challenge is the efficient implementation of algebraic effects in mainstream programming languages. Recent developments, such as OCaml 5.x, demonstrate that effect handlers can be integrated into a production language. However, OCaml's approach does not include effect typing, highlighting a remaining gap between practical implementations and more expressive type systems such as those found in Koka. Another challenge is the integration of effect systems with existing language features, such as polymorphism and concurrency.

Addressing these questions will be crucial for advancing both the theory and practice of programming languages, and for developing tools that enable more robust and compositional reasoning about effectful programs.

Updated Implementation Proposal

For my implementation, I want to formulate an extension of ALFA to include state and algebraic effects. I will to introduce syntax and semantics for performing and handling effects. Once I've come up with a reasonable formulation, I plan to extend the evaluator we've written in homework with what I've added. As stretch goal of sorts, I could define the language extension in Agda and prove progress and preservation.

References

[1] E. Moggi, *Computational lambda-calculus and monads*. University of Edinburgh, Department of Computer Science, Laboratory for Foundations of Computer Science, 1988.

Moggi's paper addresses a fundamental mismatch between the standard λ-calculus and real programs: $\beta\eta$-equivalence treats programs as pure functions, which erases important computational behavior such as nontermination, side effects, and nondeterminism. This makes it unsuitable as a foundation for reasoning about effectful programs. His solution is to introduce λ_c, the computational λ-calculus, based on a categorical semantics of computation using monads. The key idea is to distinguish values from computations, interpreting programs as morphisms $A \to TB$, where T captures computational effects. He formalizes this using monads (T, η, μ) and interprets programs in the Kleisli category, where composition corresponds to sequencing effectful computations. Moggi defines the calculus, gives a denotational semantics in categorical models, and proves that the system is sound and complete with respect to this semantics. He also demonstrates that many different effects (e.g., nondeterminism via powersets, state via store-passing functions) fit uniformly into this framework. A key limitation is that while monads provide a uniform abstraction for effects, they do not expose the algebraic structure of individual operations, making it harder to reason about or combine effects modularly. This motivates later work on algebraic effects.

[2] "Haskell 2010 Language Report." [Online]. Available: https://www.haskell.org/onlinereport/haskell2010/haskellch13.html

The Haskell report addresses the problem of providing a standard, practical interface for structuring effectful computations in a real purely-functional language. While monads had been introduced theoretically and used experimentally, a consistent language-level abstraction was needed. Its solution is the Monad type class, which defines return and >>= (bind), along with expected laws. This provides a uniform interface for sequencing computations across many effects, including I/O, state, and exceptions. However, this abstraction comes with tradeoffs: monads can be conceptually difficult to grasp, particularly due to the indirectness of bind and the lack of explicit effect tracking in types. This can obscure program behavior, especially for newcomers to functional programming or in large codebases. Additionally, while the monad laws provide important guarantees, they are not enforced by the type system, relying instead on discipline from implementers. Overall, the design is highly influential and practically successful, but not without ergonomic and pedagogical costs. A key limitation, reflected in practice, is that combining effects requires additional abstractions such as monad transformers, which introduce complexity and can make programs harder to reason about.

[3] G. Plotkin and M. Pretnar, "Handlers of algebraic effects," in *European Symposium on Programming*, 2009, pp. 80–94.

This paper addresses a core limitation of the monadic approach: the lack of modularity when combining effects. In monadic semantics, combining effects requires constructing a new monad, which is often complex and not compositional. Their solution is to shift to an algebraic view of effects, where effects are specified by operations and equations, and to introduce handlers as a general mechanism for interpreting those operations. The key insight is that computations form a free model of an algebraic theory, and handlers correspond to homomorphisms from this free model into another model. They also show that this framework generalizes exception handling and can model many effects (state, nondeterminism, I/O, concurrency) in a uniform way. A limitation is that the work is largely theoretic and foundational. It does not yet provide a full

programming language design or implementation strategy, and issues like typing are left to later work.

[4] A. Bauer and M. Pretnar, "Programming with algebraic effects and handlers," *Journal of logical and algebraic methods in programming*, vol. 84, no. 1, pp. 108–123, 2015.

This paper addresses the gap between the theory of algebraic effects and practical programming. Earlier work introduced handlers semantically, but did not fully demonstrate how they can be used in real programs. The authors present Eff, a programming language where effects are first-class and defined as algebraic operations, and handlers interpret them. Crucially, handlers are understood as homomorphisms from free algebras, directly realizing the theory from Plotkin and Pretnar in a programming setting. The paper also explicitly notes an important theoretical point: while every algebraic theory gives rise to a monad, the operations cannot be recovered from the monad alone, a key justification for the algebraic approach. The work makes a compelling case that algebraic effects and handlers offer a more modular and expressive alternative to monads for structuring effectful programs. By separating the declaration of effects from their interpretation, Eff enables a style of programming where effects can be composed, reused, and reinterpreted with significantly greater flexibility than is typical in monadic designs. This often results in clearer and more direct code, particularly when multiple effects interact, avoiding the layering and boilerplate associated with monad transformers. Compared to algebraic effects as implemented in OCaml (e.g., in Multicore OCaml), Eff presents a more fully realized and principled integration of the theory, where effects are central and uniformly handled via explicit handlers. OCaml's approach, while practical and performant, is more conservative: it integrates effects into an existing language with an emphasis on runtime efficiency and compatibility, and in some cases exposes lower-level control (such as delimited continuations) that can make reasoning more operational and less algebraically structured. At the same time, OCaml benefits from a mature ecosystem, compiler optimizations, and real-world usability that Eff lacks as a research language. This contrast highlights a broader tradeoff between theoretical clarity and practical adoption. Limitations include limited discussion of performance and compilation strategies, as well as the challenge of integrating these ideas into mainstream languages.

[5] P. Wadler, "The essence of functional programming," in *Proceedings of the 19th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, 1992, pp. 1–14.

Wadler's paper tackles the problem of how to structure effectful programs in a purely functional language without losing modularity or requiring pervasive changes to code. As he shows through examples, adding features like errors, state, or output to a pure interpreter typically requires rewriting large portions of the program. Building directly on Moggi, Wadler's solution is to internalize monads as a programming abstraction. He shows how to systematically transform programs so that computations have type $a \to Mb$, and sequencing is handled using bind. The paper demonstrates that by changing the monad, one can add new effects (exceptions, state, nondeterminism) with only local modifications to the code. The evaluation is primarily through detailed program transformations. Wadler walks through a concrete interpreter and shows how different effects can be incorporated by changing the monad definition rather than rewriting the interpreter structure. He also connects monads to continuation-passing style, showing a deeper theoretical grounding. The main limitation is that while monads improve modularity compared to naive approaches, they still impose a global structure on effects. Combining multiple effects requires layering monads (later via transformers), which can complicate code and obscure which effects are actually being used.

[6] D. Leijen, “Koka: Programming with row polymorphic effect types,” *arXiv preprint arXiv:1406.2061*, 2014.

Leijen addresses a major limitation of monadic programming: types do not clearly indicate what effects a function performs. Even in Haskell, a function of type Int -> Int may still diverge or throw exceptions. The solution is a language where effects are part of the type system, using row polymorphism to allow flexible and compositional effect typing. Function types explicitly include an effect annotation, and these effects are inferred automatically. For example, a function that prints has an io effect. A particularly strong result is that effect types are not purely static, they have semantic meaning, corresponding to all possible behaviors a given program may exhibit, enabling correctness guarantees about program behavior. The design of Koka presents a significant advance in making effects explicit, precise, and compositional within the type system. By using row polymorphism, it avoids the rigidity that often accompanies effect systems, allowing functions to remain generic over the effects they perform while still exposing meaningful information about their behavior. This leads to a notable improvement over monadic approaches, where effect information is often coarse-grained or implicit, and overcomes a key criticism of languages like Haskell. Limitations include increased type system complexity and challenges in integrating such systems into existing languages and ecosystems.